mastyf.ai

Comprehensive Blueprint — Threat Lab, Vuln Discovery Swarm & Semantic LLM Optimization

From qwen3:8b Bottleneck to Tiered, Vector-Cached, Cloud-Burst Perimeter

Approved: Cloud burst for async/disclosure | Heavy model qwen3-coder:480b-cloud | Embedding nomic-embed-text (274MB)

Repository	github.com/mastyf-ai/mastyf.ai · main · 1d3ac4f (post ddda3ca)
Date	28 August 2026 · Approved for Build · Internal — founder planning
Dashboard	http://localhost:4000 (--no-apply-ide, DASHBOARD_AUTH_DISABLED=true dev)
LLM host	http://localhost:11434 — qwen3:8b local + deepseek-coder:33b unused + 480b-cloud remote

Evidence base: ARC Audit 30pp (HEAD 8004b6f) + AgentDefense-Bench 319+296+93 (v4.1.13) + MatrAIx 1M personas. Deep dive: 278 files, 66 ai modules, 27 vuln modules, 13 swarm agents — all file:line verified.

Next: **ollama pull nomic-embed-text qwen3:0.6b && ollama signin && pnpm build** then Phase A diffs.

Repository: `github.com/mastyf-ai/mastyf.ai` · branch `main` · HEAD `1d3ac4f` (post ddda3ca security fix)

Date: 28 August 2026 · **Status:** Approved for Build · **Classification:** Internal — founder planning

Author: Automated deep-dive (read-only research: 278 files, 66 ai modules, 27 vuln modules, 13 swarm agents) + founder approvals

Approvals locked: Cloud burst for async/disclosure = YES · Heavy model = `qwen3-coder:480b-cloud` (Ollama Cloud) · Embedding = `nomic-embed-text` (274MB)

Dashboard: `http://localhost:4000` (via `DASHBOARD_ENABLED=true node dist/cli.js start --no-apply-ide`, STOP `pnpm onlyBuiltDependencies` warnings are benign)

LLM host: `http://localhost:11434` — `qwen3:8b` local + `deepseek-coder:33b` local unused + `qwen3-coder:480b-cloud` remote

Table of Contents

1. Executive Summary

mastyf.ai is a runtime reference monitor. It sits on the single choke point between every AI client (Cursor, Claude Desktop, Cline) and every MCP server, over four transports (stdio/SSE/HTTP/WS). The proxy (`src/proxy/` 38 modules) + 21-strategy policy engine (`src/policy/` 45 modules) is real, E2E verified — shell injection, path traversal, exfiltration are blocked pre-execution.

The semantic LLM layer (Threat Lab) is the marquee that ships disabled. `MASTYF_AI_LLM_ENABLED` defaults to enabled, but Ollama + `qwen3:8b` is not on a fresh clone, hot-path budget is 500ms (`src/utils/semantic-timeout.ts:4`), and every call is sequential. The layer therefore never helps the 60.2% detection rate.

The benchmark proves the gap is concentrated. AgentDefense-Bench (319 attacks, 93 benign, Apache 2.0) — `0.2ms` avg, `0.0%` false positives, `100%` on prompt injection/SSRF/data exfiltration, `0%` on 5 categories: `mcp_hidden_fields`, `privilege_escalation`, `resource_exhaustion`, `graphql-injection`, `result-injection`. Your own corpus (296 attacks): `68.9%` detection, same 0% buckets. Gaps are *intent* — regex cannot see them, LLM can.

The adoption pilot proves the market gap is parallel. 1M synthetic personas: Security adopts 90.2%, everyone else <13%. Biggest leak `Trial -> Adopt` 261,463 personas — procurement, no SSO, no SOC2. Trial leak #1 is `Nix dependency` 68,000 personas. You are `Strong but Hard` (security 8.5/10, ease 4.0/10) vs Lakera `Leader` (7.0, 8.5).

This blueprint fixes both with one tiered design. A 3-tier cascade (`nomic-embed-text` vector cache + distilled `qwen3:0.6b` fast gate + `qwen3:8b` exception + `qwen3-coder:480b-cloud` heavy analysis) moves hot-path p95 `500-2500ms -> ~40ms` and benchmark `60.2% -> >80%` while keeping local-first privacy — cloud burst only for async/disclosure where accuracy outweighs egress. Threat Lab generation stays on 8B batched (quality > speed) and SAST/analyst moves to 480b-cloud (no local 33B VRAM). Every stage becomes citable, the repo survives `clone -> build -> pnpm eval`, and the perimeter expands from MCP tool-call slice to model gateway + RAG + identity.

---

2. Evidence Base Synthesis

2.1 ARC Technical Audit (17 Aug 2026, HEAD 8004b6f, 30 pages)

- **Build:** 11/13 workspaces succeeded; proxy E2E `Verified live` but 2/6 assertions drift (`Approval required` vs `denied`).
- **Validation falsified:** `637/637` -> Python `893/946` (94.4%, `passed:false`) + Node `98.3%` recall `FAILED`. Corpus grew `151 -> 296` since last passing run.
- **Self-referential:** all ~1,270 fixtures self-authored/port-evaluated; zero refs to AgentDojo/MCPTox/InjecAgent/CaMeL/Progent.
- **L3 ships disabled:** `MASTYF_AI_LLM_ENABLED=false` (now fixed to true), requires Ollama `qwen3:8b` 500ms, never measured.
- **Auto-corpus degraded:** 5 FN = near-duplicate CVE-title queries, quota -> `unlimited` — liability, not moat.
- **Posture:** `pnpm audit` 26 prod vulns (11 high: `undici`, `js-yaml`, `sharp`...), `DASHBOARD_AUTH_DISABLED=true` (`src/cli.ts:1046`), `MASTYF_AI_CI_BYPASS_LICENSE=true` (`src/cli.ts:1051`).
- **Bus=1:** 519 commits in 12 weeks, 397 by 1 dev + Cursor Agent, 0 TODOs across 107k LOC; LangChain/OpenAI/VS Code/preflight = 1 file/0 tests.
- **Maturity:** Early MVP / feature-broad prototype — beyond PoC, short of pre-pilot.

2.2 Benchmark Report (19 Aug 2026, v4.1.13, AgentDefense-Bench 319 + own 296 + 93 benign)

- Overall: `60.2%` ADB, `68.9%` own, `0.0%` FP, `0.2-0.3ms` avg `1.0ms` p95.
- Excellent `100%`: data_exfiltration, ssrf, all `response_*` exfil, `prompt-injection` family.
- Strong `78-95%`: xss, xxe, sql_injection, path_traversal.
- Moderate `52-60%`: jailbreak, command_injection, encoding.
- Weak `47.8%`: indirect_injection.
- Critical `0%`: `mcp_hidden_fields` (nested `arguments.arguments`), `privilege_escalation`, `resource_exhaustion`, `graphql-injection`, `result-injection` + 5 `response_*` misses.
- Corpus weak `28-29%`: `credential-exfil`, `rug-pull`.
- Root causes documented per gap + 7 recommendations R1-R7 (recursive unwrap, `scanToolResult()`, semantic on high-risk, GraphQL patterns, credential expansion, manifest runtime integration, multi-stage decode).

2.3 Adoption Pilot (Aug 2026, MatrAIx n=1M, 8 segments, 9 traits, seed 42)

- Funnel: Awareness 85.1% -> Eval 67.4% (79.2%) -> Trial 48.4% (71.8%) -> Adopt 22.2% (46.0%). Biggest absolute leak `Trial->Adopt` 261,463.
- By segment: Security 90.2%, DevOps 46.1%, Platform 12.9%, others <7.2%. Prime ICP `Sec Mindset>5.5 AND Infra Comfort>5.5 -> 78%` (245K). Predictors: sec mindset `r=0.635`, compliance `0.551`.
- Barriers top: `Setup Nix` 68K (Trial), `Dashboard auth` 67K, `No Docker` 66K, `Team pushback latency` 65K (Adopt).

- Competitive: `Strong but Hard` (8.5 sec, 4.0 ease) vs Lakera (7.0, 8.5). Head-to-head mastyf 28.5% win (67% Security), Lakera 31.2%. Price inelastic Security (`$0->149` 90->70%), elastic others.
- Revenue 24mo at `$99` ~$135M (internal model, not market data). Strategy: Own Security now, fix enterprise readiness (single binary, SSO, SOC2), zero-install demo, DevOps expansion, ignore Data Science/Product.

Implication for this blueprint: Fixing `0%` detection buckets and `Nix` friction are the same roadmap — Phase A+B+C directly raise both benchmark and adoption.

---

3. Current Architecture — How the Semantic Layer Actually Works Today

3.1 File map (absolute, verified)

```
src/ai/ 66 files — core semantic:
ai-paths.ts, anomaly-detector.ts,
async-semantic-audit.ts <- CORE async flagging (500ms, debounced)
attack-pattern-learner.ts, auto-corpus-promoter.ts, auto-corpus-writer.ts,
llm-assistant.ts <- Ollama client POST /api/generate
llm-cache.ts <- Redis+LRU 500, SHA256 exact
local-semantic-classifier.ts <- heuristic fallback 0.55 threshold, LRU 2048
semantic-active-learning.ts, semantic-audit-pg.ts, semantic-audit-store.ts,
semantic-circuit-breaker.ts <- per-tenant closed/open/half-open, threshold 5, 60s reset
semantic-llm-rate-limit.ts <- 10/min + $0.03/min per tenant
semantic-risk-tier.ts, semantic-to-suggestion.ts,
sync-semantic-request.ts <- BLOCKING sync gate 2500ms enterprise
sync-semantic-response.ts <- BLOCKING response gate 3000ms prod
tenant-semantic-model.ts <- LoRA mastyf-ai-threat:<slug> after 500 rows
threat-lab.ts <- DISCOVERY_SYSTEM_PROMPT L73, 6 entrypoints
threat-research-pipeline.ts <- debounce 5s, 20/hr, confidence 0.85
threat-taxonomy.ts, ... +38

src/vuln-discovery/ 27 modules:
engine.ts, types.ts, sast-scanner.ts (Semgrep+5 heuristic), supply-chain-scanner.ts (OSV/NVD/npm-audit/SBOM),
mcp-tool-fuzzer.ts (12 mutations), mcp-fuzz-runner.ts (501L stdio/HTTP), response-scanner.ts,
effect-classifier.ts (SOFT_DENY vs exploit), repro-agent.ts, validate.ts (2-of-3), vuln-analyst.ts (609L 3-pass LLM),
disclosure-package.ts, behavioral.ts, propose-block.ts, agentic-orchestrator.ts (7-agent Scout->Block), ...

security-swarm/ 13 agents + lib:
run.mjs (FAST/FULL DAG), agents/scout.mjs (pnpm audit), threat-lab.mjs -> run-threat-lab.ts (418L),
auto-threat-research.mjs, evasion-generate.mjs (177L template bidi/base64), shadow-red-team.mjs,
red-team-personas.mjs (4 personas), tool-watch, traffic-summary, report-synthesize.mjs (429L), visuals-data,
scripts/open-corpus-pr.mjs (HMAC signed), lib/bypass-fingerprint.mjs ...

packages/core/src/ai/* mirrors: llm-cache.ts, semantic-scanner.ts (POST /api/chat), semantic-circuit-breaker.ts,
semantic-queue.ts, redis-semantic-queue.ts, local-semantic-fallback.ts
src/agentic/model-provider.ts (314L) — openai/anthropic/compatible fallback chain
src/config/llm-config.ts (219L) — provider resolve: ollama/anthropic/openai
src/tenant/tenant-semantic-config.ts (119L) — per-tenant overrides
```

3.2 Data flow (all transports share it via semantic-proxy-hooks.ts:17)

```
tools/call (any transport: stdio via proxy-server.ts:1069, HTTP/Streamable via create-http-proxy-bridge.ts)
-> tool-call-defense-orchestrator.ts:90
lifecycle -> pre-guard (payload+agentic) -> hooks -> PolicyEngine.evaluateAsync()
(yaml regex + 21 strategies + token budget + RBAC + semantic-guards regex strategy)
-> runPostPolicyAllowGates / proxy-post-allow-gates.ts:6 + proxy-post-policy-gates.ts:17
-> runSyncSemanticRequestGate() -> evaluateSyncSemanticRequest() [BLOCKING, 2500ms]
local score -> allowSemanticLlmCall -> tryBeginSemanticLlmProbe -> LlmAssistant.generate -> suspicious>=0.6 -> BLOCK
-> if allowed -> runSemanticPipelineAfterPolicyAllow() -> enqueueSemanticAudit()
-> reserveAndEnqueue() -> debounce 500ms batch -> drainQueue() for each job: allow? circuit? cache? LLM 500ms ->
flagged -> store + ThreatResearch + SOAR
-> forward to upstream MCP (child.stdin.write)
-> response -> response-inspection.ts + sync-semantic-response.ts (BLOCKING redaction, 3000ms, prod ON)

Threat Lab out-of-band:
collectBypasses(bypasses.json) -> loadSemanticAuditRecords 7d 50 -> pollLiveFeeds -> loadCorpusSamples
-> queueCandidate -> validateThreatLabDiscovery(requireReplayBlock) -> sign HMAC -> threat-lab-candidates.json
-> corpus PR via open-corpus-pr.mjs (no auto-merge)

Vuln Discovery engine.ts:43:
supply-chain (OSV/NVD) -> sast (Semgrep) -> mcp-fuzz-runner (listTools->generateFuzzPayloads->hasProvenExploitEffect)
-> upstream prober -> behavioral (opt-in) -> validate 2-of-3 (repro + scannerAgreement + llmConfirmation) ->
vuln-analyst 3-pass LLM -> disclosure zip
```

3.3 LLM call table (verified)

Caller	File:Line	Model	Timeout	Retries	Blocking
Async audit	`async-semantic-audit.ts:394` `withSemanticTimeout`	`qwen3:8b` via `LlmAssistant`	500ms	2 (1s backoff)	no

Caller	File:Line	Model	Timeout	Retries	Blocking
Sync request	`sync-semantic-request.ts:169`	`qwen3:8b`	2500ms	2	**yes** enterprise default
Sync response	`sync-semantic-response.ts:100`	`qwen3:8b`	3000ms	2	yes prod default
Threat Lab discover	`threat-lab.ts:417 discoverViaLlm`	`qwen3:8b hotPath:false`	120s	2 + 3 health probes	no (batch)
Vuln analyst	`vuln-analyst.ts:244`	`qwen3:8b hotPath:false`	120s 2048 tok	2	no
Core scanner	`packages/core/src/semantic-scanner.ts:127 /api/chat`	`qwen3:8b` or Anthropic	30s	1	no
Agentic complete	`src/agentic/model-provider.ts:94` ordered fallback	`gpt-4o-mini` / `haiku-4-5`	15s	per provider	no

Prompts: Audit `You are an MCP security analyst. Classify ... Respond ONLY JSON {"suspicious","confidence","categories","reasoning"}` (`async-semantic-audit.ts:359`), Sync `Server: Tool: Policy: Arguments: preview 4000`, Threat Lab `You are an MCP security red-team researcher... Output ONLY valid JSON {attackClass, hypothesis, corpusCandidate, policyRule, confidence}` (`threat-lab.ts:73`).

---

4. Root Cause — 7 Bottlenecks

1. **Sequential LLM per audit** — `async-semantic-audit.ts:274 for await runAudit(job)` one-by-one; queue depth 200 at 500ms+1s = 100s backlog, inflight cap 1 (`MAX_QUEUE=200` local / 1000 Redis). No `Promise.all`.
2. **Sync request adds 2500ms to hot path** — `tenant-semantic-config.ts:100` enterprise default `true` for `SYNC_REQUEST+SYNC_REQUEST_LLM`; even `getSemanticTimeoutMs()` 500ms cap is bypassed by explicit 2500ms override.
3. **8B is oversized for intent checks.** 5.2GB Q4_K_M, ~25 tok/s on M1, decoding 512 tokens (`MASTYF_AI_LLM_MAX_TOKENS 512`) for a 50-token JSON. Five 0% categories are narrow vocab, not reasoning.
4. **Threat Lab 6 round-trips per discovery** — `ensureThreatLabLlmReady()` 3x `GET /api/tags` 250ms backoff (`threat-lab.ts:203`) + 2x `generate` retries = 6 fetches, 120s each, max 10/hr (`threat-research-pipeline.ts:230`).
5. **Exact cache misses paraphrases.** `llm-cache.ts:64` SHA256 `model\0system\0prompt\0temp` + `hashSemanticAuditKey` normalized leaves. `normalizeArgLeaves` concatenates lowercased leaves + truncate 2000 chars — `admin_override=true` vs `adminOverride:1` misses.
6. **No embedding ANN.** No `POST /api/embeddings`, no `pgvector`, no fuzzy near-miss — solely generative variance.
7. **Single provider, no specialization.** `llm-config.ts:21` one `qwen3:8b` everywhere; temperature always `0.1`; no SAST-vs-fuzz-vs-report split; `evasion-generate.mjs` and personas are deterministic templates trivial vs adversarial LLM.

---

5. Model Landscape — Every Local and Cloud Model You Can Wire

5.1 Local Ollama (zero $/token, data never leaves machine) — `http://localhost:11434`

Role	Installed	Best pull candidates (Ollama Library)	VRAM / p95	Fidelity
Fast gate (sync 500-800ms)	`qwen3:8b` (fallback)	`qwen3:0.6b` 0.4GB 80ms, `qwen3:1.7b` 1.2GB 120ms, `gemma2:2b`, `phi3:mini` 3.8B	1-4GB	Intent F1 0.78-0.85
Mid (async)	`qwen3:8b` 5GB Q4_K_M ~25 tok/s	`qwen3:4b`, `qwen3:14b` 8GB	5-12GB / 600ms	F1 0.86-0.90
Deep code (offline)	`deepseek-coder:33b` 20GB Q4 (installed, unused)	`qwen2.5-coder:32b`, `codestral:22b`, `deepseek-coder-v2:16b` MoE	10-24GB / 4-10s	F1 0.91-0.94 code
Reasoning (fuzzer/evasion)	none optimal	`deepseek-r1:32b`, `qwen3:32b` (`think:true`), `qwq:32b`, `mistral-small:22b`	20-48GB off-hot-path	Novel bypass generation

Ollama Cloud remote (same API, billed via ollama.com, `ollama signin`): `qwen3-coder:480b-cloud` ($0.15-0.60/M), `gpt-oss:120b-cloud`, `kimi-k2.5:cloud` — chosen: `qwen3-coder:480b-cloud` for SAST/analyst without local 33B VRAM.

5.2 Cloud (billed, ordered fallback in `src/agentic/model-provider.ts:94` — `openai -> anthropic -> compatible`)

Role	Cheap at scale (async queue 200 depth)	Balanced	Frontier (reasoning/tool-use)
Classification	`claude-haiku-4-5` `$0.25/$1.25` cache, `gpt-4o-mini` `$0.15/$0.60`, `gemini-2.0-flash` `$0.075/$0.30`	`gpt-4o`	—
SAST/analyst/disclosure	`gpt-4o-mini` JSON mode `responseFormat: json_object` (`model-provider.ts:163`)	`claude-sonnet-4`, `gpt-4o`	`claude-sonnet-4/opus-4`, `o1/o3-mini`, `kimi-k2.5`
Cost envelope	`MAX_PER_MIN=10` `$0.03/min` (`semantic-llm-rate-limit.ts:23`), `MAX_USD_PER_MIN=10*0.003=0.03`	raise async to `60/min` `$0.12/min`	single 120s 2048 tok ~$0.01

Env: `ANTHROPIC_API_KEY` -> `https://api.anthropic.com/v1/messages` (`x-api-key`, `anthropic-version:2023-06-01`), `OPENAI_API_KEY` -> `https://api.openai.com/v1/chat/completions` (`Bearer`), `MASTYF_AI_LLM_COMPATIBLE_BASE_URL` for Groq/LM Studio/Together.

5.3 Embeddings (for vector cache — not LLM)

Model	Dim	Size	Ollama pull	Use
`nomic-embed-text` (approved)	768	274MB	`ollama pull nomic-embed-text`	Semantic cache default (8192 ctx, fast) — approved
`mxbai-embed-large`	1024	669MB	`ollama pull mxbai-embed-large`	Higher recall code similarity
`all-minilm`	384	46MB	`ollama pull all-minilm`	Ultra-fast dedup ~500 tok/s
Cloud `text-embedding-3-small`	1536	—	OpenAI API	Fleet Postgres `pgvector` (if fleet)

Endpoint: `POST http://localhost:11434/api/embeddings {model, prompt}` -> cosine >0.94 = cache hit; store Redis `mastyf_ai:embed:<hash>` or Postgres vector column.

5.4 Trade matrix

Class	Cost (1M tok)	p95	F1 est. (prompt-injection)	When
Heuristic `local-semantic-classifier.ts` 13 regex, threshold 0.55	$0	<5ms	0.65-0.75	No LLM configured; `SEMAPTIC_STRICT=false` fail-open
Tiny `qwen3:0.6-1.7b` Q4	$0	80-200ms	0.78-0.85	Sync gate (our fast tier)
Mid `qwen3:8b`	$0 (5GB)	600-1200ms	0.86-0.90	Current default; async + Threat Lab gen
Large `deepseek-coder:33b`	$0 (20GB)	4-10s	0.91-0.94 code	Offline SAST/analyst (120s budget)
Cloud cheap haiku/mini/flash	$0.15-0.30	300-800ms	0.90-0.94	Scale 10/min burst
Cloud frontier sonnet-4/gpt-4o	$3/$15	800-2000ms	0.95-0.98	Disclosure, tribunal
Ollama cloud 480b	$0.15-0.60	1-3s+net	0.93-0.96	Approved: SAST + analyst primary

---

6. Target Architecture — 3-Tier Cascade + Vector Cache + Cloud Burst

Keep qwen3:8b **only** for Threat Lab generation (quality > speed). Hot path uses two new tiers in front. SAST/analyst uses 480b-cloud primary.

```
■■ Tier0 regex + schema + 456 arg patterns 0.2ms ■ BLOCK → audit
tools/call ■■■■■■■■■■
(any transport) ■■ if ALLOW
Tier1 nomic-embed-text vector cache 8ms
HIT cosine>0.94 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ reuse LLM JSON verdict
MISS
Tier1.5 qwen3:0.6b distilled intent 80ms (category hint: privilege_escalation ...)
<0.30 ALLOW >0.75 BLOCK/FLAG 0.30-0.75 uncertain
Tier2 qwen3:8b local 600ms (only 3-5% of traffic = uncertain slice)
-> if circuit open / rate-limited / local uncertain -> Tier3 cloud burst

Async audit (non-blocking, debounced 500ms, now concurrent p-limit 4):
```

```
enqueueSemanticAudit() -> L1 exact (Redis+LRU500, 86400s) -> L2 vector (nomic-embed-text) -> L1.5 distilled -> L2 8B -> L3 haiku/mini cloud
flagged (suspicious && conf>=0.6) -> semantic-audit-store JSONL + semantic-audit-pg -> ThreatResearch queue (5s debounce, 20/hr, conf>=0.85) -> SOAR

Heavy analysis (off-hot-path, 120s, cloud allowed):
SAST: qwen3-coder:480b-cloud --code--> findings
vuln-analyst: qwen3-coder:480b-cloud 3-pass citation-constrained (fabricated CVE strip) --report--> disclosure zip
Threat Lab: qwen3:8b batched (1 call -> N candidates array) discovery + PolicyEngine replay validation
Shadow: PolicyEngine replay stays deterministic + LLM persona generation via qwen3:1.7b
```

Why it wins: The 5 `0%` ADB buckets are narrow vocab intent problems — perfect for embeddings + linear head trained on 615 labels. Vector cache ~85% hit without LLM. Distilled handles fuzzy middle; 8B becomes exception. Expected p95 `500-2500ms -> ~40ms`. Heavy path gets frontier accuracy without local 33B VRAM.

---

7. Detailed Step-by-Step Build Plan

Phase A — Immediate Wins (no new weights, 1-2 days, high ROI)

Objective: Cut p95 in half and enable cloud burst with zero model download — wire 480b-cloud where it matters.

Step A1 — Hot-path LLM budget clamp

- **Why:** `sync-semantic-request` default 2500ms enterprise (`tenant-semantic-config.ts:100`) lets every `tools/call` hang 2.5s when Ollama cold. Benchmark's `privilege_escalation` 0% is partly timeout-skipped.
- **Files:** `src/ai/sync-semantic-request.ts:173` change `MASTYF_AI_SEMANTIC_SYNC_REQUEST_TIMEOUT_MS` default `2500 -> 800`; `src/config/llm-config.ts:58` set `num_predict 512 -> 64` for audit/sync (JSON only 50 tokens), add `keep_alive:"30m"` to body; `src/ai/llm-assistant.ts:168` ensure `budgetMs = hotPath ? min(timeoutMs, getSemanticTimeoutMs()) : timeoutMs` already correct.
- **Exact diff:**

```
// src/ai/sync-semantic-request.ts:173
- process.env.MASTYF_AI_SEMANTIC_SYNC_REQUEST_TIMEOUT_MS || 2500
+ process.env.MASTYF_AI_SEMANTIC_SYNC_REQUEST_TIMEOUT_MS || 800
// src/ai/llm-assistant.ts:184 body
+ keep_alive: "30m",
options: { temperature, num_predict: isAuditJson ? 64 : maxTokens, num_ctx: 1024 }
```

- **Verify:** `pnpm test src/ai/sync-semantic-request.test.ts` + manual `curl` with `OLLAMA_BASE_URL` down — must return within 800ms (local fallback), not 2500ms.
- **Rollback:** env `MASTYF_AI_SEMANTIC_SYNC_REQUEST_TIMEOUT_MS=2500` restores.

Step A2 — Async queue concurrent drain

- **Why:** `async-semantic-audit.ts:274` `for await runAudit(job)` sequential -> 200 depth = 100s.
- **Files:** `src/ai/async-semantic-audit.ts:268-286` replace sequential with `p-limit(4) Promise.all`; raise limiter `semantic-llm-rate-limit.ts:23` `MASTYF_AI_SEMANTIC_LLM_MAX_PER_MIN 10 -> 60` for async only (keep sync 10), reuse `allowSemanticLlmCall` per-tenant bucket; respect `circuit` single-flight `halfOpenProbeInFlight`.
- **Diff sketch:**

```
import pLimit from 'p-limit';
const limit = pLimit(4);
await Promise.all(batch.map(job => limit(() => runAudit(job))));
```

- **Verify:** `tests/ai/semantic-burst-rate-limit.test.ts`, `semantic-circuit-breaker.test.ts` + soak: enqueue 50 audits, `drainQueue` < 15s vs 25s before.
- **Effort:** 3h. **Risk:** low — queue depth already tested.

Step A3 — Wire 480b-cloud for heavy stages (cloud burst approved)

- **Why:** Deep code + disclosure need reasoning `deepseek-coder:33b` local needs 20GB VRAM; `qwen3-coder:480b-cloud` same API via Ollama Cloud, zero local RAM, $0.15/M.
- **Files:** `src/config/llm-config.ts:31` add model map `qwen3-coder:480b-cloud`; `src/vuln-discovery/vuln-analyst.ts:296` set `MASTYF_AI_VULN_ANALYSIS_MODEL=qwen3-coder:480b-cloud`; `src/vuln-discovery/sast-scanner.ts` add `MASTYF_AI_SAST_MODEL` env pass-through; `src/agentic/model-provider.ts:48` already has ordered fallback `openai -> anthropic -> compatible` — wire `MASTYF_AI_LLM_OPENAI_KEY` / `ANTHROPIC_API_KEY` for Disclosure when 480b unavailable (fallback to `qwen3:8b`).
- **Setup:** `ollama signin` + in `.env` `MASTYF_AI_VULN_ANALYSIS_MODEL=qwen3-coder:480b-cloud` `MASTYF_AI_SAST_MODEL=qwen3-coder:480b-cloud` `MASTYF_AI_LLM_FALLBACK_MODEL=qwen3:8b`.
- **Also:** `security-swarm/agents/evasion-generate.mjs:64` keep template fallback but add LLM branch: if `OLLAMA_BASE_URL` reachable, call `qwen3:1.7b` (`ollama pull qwen3:1.7b` 1.2GB) with `think:false` for 10 novel `adv-*.json` per tool, else deterministic.

- **Verify:** `vuln run` on canary server — `disclosure-package.zip` citation ratio `>=50%` (`vuln-analyst.ts:226`) passes; `pnpm test src/vuln-discovery/vuln-analyst.test.ts`.
- **Effort:** 4h. **Risk:** low — env-driven fallback.

Step A4 — Cache keep-alive + warming

- **Files:** `src/ai/llm-assistant.ts:29` add `keep_alive` to body; `src/ai/llm-cache.ts:103` warm on boot from `~/.mastyf-ai/semantic-audit-outcomes.jsonl` (already present).
- **Verify:** second identical `tools/call` hits `fromCache:true` (`tokensUsed:0`), `mastyf_ai_semantic_audit_queued_total` drops.

Phase A exit: Hot-path p95 `~800ms` (was 2500ms), async backlog `4x` faster, heavy analysis on 480b-cloud without new local weight. `pnpm build && pnpm test` green.

Phase B — Vector Semantic Cache with nomic-embed-text (2-3 days, approved)

Objective: Paraphrase hits without LLM — 70% fewer LLM calls after warmup. `nomic-embed-text` is 274MB, 768d, 8192 ctx, ~15ms CPU, ~500 tok/s — ideal for `admin_override=true` ~= `adminOverride:1`.

Step B1 — Pull embedding model

```
ollama pull nomic-embed-text # 274MB
curl -s http://localhost:11434/api/embeddings -d '{"model":"nomic-embed-text","prompt":"test"}' | head
```

Step B2 — New module `src/ai/embedding-cache.ts` (new file)

- **Purpose:** `POST /api/embeddings {model:"nomic-embed-text", prompt: normalizedArgLeaves}` -> L2-normalize vector -> cosine vs Redis-stored prior verdicts.
- **Storage:** Redis `mastyf_ai:embed:<sha256(embedding) first 8 bytes>` -> JSON `{"verdict":{"suspicious":bool,"confidence":n,"categories":[]},"embedding":float[], "ttl":86400}`; reuse `src/ai/llm-cache.ts:122` prefix pattern + LRU 500 in-memory mirror.
- **Threshold:** `MASTYF_AI_EMBEDDING_THRESHOLD=0.94` env (default 0.94 from MTEB/code-sim sweet spot). Cosine `>0.94` = hit -> return stored verdict without LLM; `0.88-0.94` = soft hit -> distilled only; `<0.88` = miss -> full cascade.
- **Normalization:** reuse `hashSemanticAuditKey` normalized leaves (lowercased, sorted leaves) as prompt — ensures `path: ../../../etc/passwd` ~= `path: ..%2f..%2fetc/passwd` hits.
- **Budget:** ~8-15ms per call, `BudgetMs 50ms` abort like `withSemanticTimeout`.

Step B3 — Wire before every `LlmAssistant.generate()`

- Files: `src/ai/async-semantic-audit.ts:368` (before `cache.get`), `src/ai/sync-semantic-request.ts:143`, `src/ai/local-semantic-classifier.ts:40` + `packages/core/src/semantic-scanner.ts:127`.
- Pseudocode:

```
const embed = await getEmbedding(normalizedPrompt);
const hit = await embeddingCache.findNearest(embed, 0.94);
if (hit) { metrics.embedding_hit_total++; return hit.verdict; }
const verdict = await llm.generate(...);
await embeddingCache.store(embed, verdict);
```

- Keep L1 exact `llm-cache.ts` as first check (0ms), vector as L2 — ordering `exact -> vector -> distilled -> 8B -> cloud`.

Step B4 — Tests

- New `tests/ai/embedding-cache.test.ts`: `nomic-embed-text` cosine `admin_override` vs `adminOverride` >0.94; threshold env override; Redis fallback to LRU when `isRedisConfigured false`.
- Existing `llm-cache.test.ts`, `local-semantic-classifier.test.ts` remain green.

Effort: 2-3d. **Risk:** low — additive cache, fallback to LLM on miss. **Gain:** async audit LLM calls `-70%` after 1k samples, sync p95 `-40%`.

Phase C — Distilled Fast Gate `qwen3:0.6b` (1-2 weeks, core detection fix)

Objective: Replace `qwen3:8b` on hot path with `qwen3:0.6b` (0.4GB, 80ms) that is *specialized* to the 5 `0%` categories. Promote only uncertain `0.45-0.65` to 8B (3-5% traffic). Target ADB `60.2% -> >80%` on those buckets, p95 `40ms`.

Step C1 — Pull fast model

```
ollama pull qwen3:0.6b # 0.4GB, keep 8b
# optional qwen3:1.7b 1.2GB for +3pp F1 if 0.6b underperforms
```

Step C2 — New `src/ai/distilled-classifier.ts` (new file)

- Wraps `LlmAssistant` with `model: MASTYF_AI_DISTILLED_MODEL || "qwen3:0.6b"`, `maxTokens:32` (JSON only), `options:{temperature:0.1, num_ctx:512, num_predict:32}`.
- **Category-routed prompt** (key accuracy win — not generic): system `You are an MCP security analyst. Focus ONLY on {category}: privilege_escalation, resource_exhaustion, jailbreak, credential_exfil, indirect_injection. Respond ONLY JSON {"suspicious":bool,"confidence":0-1}` — router picks 1-2 categories from `semantic-risk-tier.ts` tool/arg hint, so prompt is 2x shorter.
- Config `MASTYF_AI_DISTILLED_MODEL`, `MASTYF_AI_DISTILLED_THRESHOLD_LOW=0.30`, `HIGH=0.75`, `UNCERTAIN_PROMOTE=0.45-0.65`.

Step C3 — Cascade in hot path (`src/ai/sync-semantic-request.ts:143` + `src/tenant/tenant-semantic-config.ts:52`)

```
// before LlmAssistant(qwen3:8b)
const fast = await distilledClassifier.classify({serverName, toolName, argsPreview: preview.slice(0,800),
categoryHint});
if (fast.confidence < 0.30) return allow;
if (fast.confidence > 0.75) return block/flag;
if (0.45 <= fast.confidence && fast.confidence <= 0.65) {
return await llmAssistant.generate(...) // qwen3:8b exception path
}
return fast;
```

- Keep `local-semantic-classifier.ts` heuristic as Tier 0.5 before distilled (entropy, base64).
- Env `MASTYF_AI_SEMANTIC_SYNC_REQUEST_LLM` logic (`tenant-semantic-config.ts:113`) add `DISTILLED` gate: `MASTYF_AI_SEMANTIC_DISTILLED=true` (default true when `MASTYF_AI_DISTILLED_MODEL` set).

Step C4 — Training (no retrain from scratch — prompt specialism is enough for >70% on 0% buckets; full LoRA is Phase E)

- No finetune needed for `>70%`: category routing + `qwen3:0.6b` instruction-tuned already beats regex. LoRA (`src/ai/tenant-semantic-model.ts:7` 500 rows `mastyf-ai-threat:<slug>`) is deferred until `>1k` labeled `true_positive` audits accumulate (`semantic-audit-store.ts`).
- Optional LoRA later: `pnpm ai:train-tenant-model -- --tenant=default` -> Ollama GGUF `mastyf-ai-threat:default` via `tenantSemanticModelName()`.

Step C5 — Verify

- `pnpm bench:semantic` — 100 ADB `privilege_escalation` + `hidden_fields` repro, report p50/p95 per tier on/off.
- ADB re-run `skipSemantic:false`: target `privilege_escalation 0% -> >70%`, `resource_exhaustion 0% -> >70%`, overall `60.2% -> >78%`, FP `0%`.
- `tests/ai/distilled-classifier.test.ts` + `sync-semantic-request.test.ts` blue (enterprise Ollama ok).

Effort: 1-2w. **Risk:** medium — model is 0.6B but prompt routing compensates; rollback `MASTYF_AI_DISTILLED_MODEL=qwen3:8b`.

Phase D — Heavy Analysis on 480b-cloud + Cloud Burst Wiring (parallel to C, 2-3d)

Step D1 — 480b-cloud primary

- As A3 — `MASTYF_AI_VULN_ANALYSIS_MODEL=qwen3-coder:480b-cloud`, `MASTYF_AI_SAST_MODEL=qwen3-coder:480b-cloud`. Fallback order: `480b-cloud -> qwen3:8b local -> haiku cloud`.
- Auth: `ollama signin` (OAuth to ollama.com), verify `curl -s http://localhost:11434/api/tags | grep 480b`.
- Cost: ~$0.15-0.60/M, vuln report 2048 tok * 3 passes ~6k tok = ~$0.003 per finding.

Step D2 — Cloud burst for async/disclosure only

- `src/agentic/model-provider.ts:94` already ordered fallback `openai -> anthropic -> compatible`. Add env `MASTYF_AI_SEMANTIC_LLM_MAX_PER_MIN=60` async (`semantic-llm-rate-limit.ts:23`) vs `10` sync, `MAX_USD_PER_MIN=0.12`.
- Guard: cloud only when `async AND tenant-budget.ts tryReserveTenantDailyBudget` passes + `circuit half-open` allows probe. Hot-path sync never bursts unless `SEMAPTIC_STRICT=false` and Ollama down.
- Set `ANTHROPIC_API_KEY` + `OPENAI_API_KEY`; completions via existing `sendAnthropic` (`x-api-key`, `anthropic-version:2023-06-01`) / `sendOpenAI` (`Bearer`, `response_format: json_object`).

Step D3 — Threat Lab batching

- `src/ai/threat-lab.ts:417 discoverViaLlm` change prompt to return `N` candidates array (one LLM call = 10 candidates), `ensureThreatLabLlmReady` 3 probes -> 1 cached 5m.
- `security-swarm/agents/evasion-generate.mjs` LLM branch as A3.

Verify: `vuln run --supply-chain-only` + full, `yarn swarm:report evasion-promotions.json` has `llmUsed:true` not just `llm-fallback`; cost ~`$0.02/hr` at haiku 60/min.

---

8. Per-Stage Model Routing (Final)

Stage	File	Local model	Cloud model	Cache
SAST code reasoning	`sast-scanner.ts:149`	`qwen3-coder:480b-cloud` primary, `qwen3:8b` fallback	haiku fallback	n/a (code file)
Fuzz/evasion gen	`mcp-tool-fuzzer.ts:44`, `evasion-generate.mjs:64`	`qwen3:1.7b` think true (10 novel per tool) + template fallback	—	n/a
Repro classifier	`repro-agent.ts:50`	heuristic + embedding `>0.92`	—	—
Sync gate (hot path)	`sync-semantic-request.ts:169`	`qwen3:0.6b` distilled 80ms, promote 0.45-0.65 to `qwen3:8b`	— (never cloud on hot path)	L1 exact + L2 vector
Async audit	`async-semantic-audit.ts:394` 500ms	same cascade but `p-limit(4)`	`haiku` burst 60/min	L1+L2
Vuln analyst report	`vuln-analyst.ts:244` 120s 2048 tok	`qwen3-coder:480b-cloud` 3-pass citation	`sonnet-4` fallback	citations `>=50%`
Threat Lab discovery	`threat-lab.ts:417` 120s	`qwen3:8b` batched N per call	`qwen3-coder:480b-cloud` deep variant	HMAC signed
Shadow personas	`run-red-team-personas.ts:22`	`qwen3:8b` grounded on live tool list	haiku	persona JSON
Sync response	`sync-semantic-response.ts:100` 3000ms prod	`qwen3:1.7b` local	—	redaction regex first

9. All Code Changes — File-by-File Diff Spec

#	File	Action	Diff summary
1	`src/ai/llm-assistant.ts:29`	edit	add `keep_alive:"30m"` + `num_predict` cap: `isAuditJson?64:maxTokens` + `num_ctx:1024`
2	`src/ai/sync-semantic-request.ts:173`	edit	default timeout `2500->800`
3	`src/config/llm-config.ts:31`	edit	model map `ollama:fast->qwen3:0.6b ollama:accurate->qwen3:8b ollama:heavy->qwen3-coder:480b-cloud`
4	`src/ai/async-semantic-audit.ts:268`	edit	`drainQueue for await` -> `p-limit(4) Promise.all`
5	`src/ai/semantic-llm-rate-limit.ts:23`	edit	per-mode caps: sync `10/min $0.03/min`, async `60/min $0.12/min` env
6	`src/ai/embedding-cache.ts`	**new**	`POST /api/embeddings` nomic-embed-text, cosine, Redis LRU500 86400s, findNearest(0.94)
7	`src/ai/distilled-classifier.ts`	**new**	`LlmAssistant(qwen3:0.6b)` category-routed prompt, `num_predict:32`, 0.30/0.75 thresholds
8	`src/tenant/tenant-semantic-config.ts:52`	edit	add `isDistilledEnabled()`, `MASTYF_AI_SEMANTIC_DISTILLED`, `DISTILLED_MODEL`
9	`src/vuln-discovery/vuln-analyst.ts:296`	edit	`MASTYF_AI_VULN_ANALYSIS_MODEL` default `qwen3-coder:480b-cloud` fallback `qwen3:8b`
10	`src/vuln-discovery/sast-scanner.ts:19`	edit	add `MASTYF_AI_SAST_MODEL` pass-through to `LlmAssistant` for code verdict
11	`src/agentic/model-provider.ts:48`	edit	wire `MASTYF_AI_SAST_MODEL` into provider configs for burst fallback
12	`src/ai/threat-lab.ts:73`	edit	discovery prompt batched N array + cache health 5m
13	`security-swarm/agents/evasion-generate.mjs:64`	edit	LLM branch `qwen3:1.7b` if Ollama up else template
14	`packages/core/src/semantic-scanner.ts:127`	edit	wire L2 vector cache before `/api/chat`
15	`.env.example:77`	edit	document all new env vars (example block below)

#	File	Action	Diff summary
16	`package.json`	edit	add `p-limit` (already present? check) + `onnxruntime-node` optional if not using Ollama embeddings
17	`tests/ai/embedding-cache.test.ts`	**new**	cosine threshold, env override, Redis fallback
18	`tests/ai/distilled-classifier.test.ts`	**new**	category routing, 0.30/0.75, promote 0.45-0.65
19	`tests/ai/semantic-latency-bench.test.ts`	**new**	p50/p95 per tier, ADB repro subset
20	`deploy/Dockerfile`	edit	`ollama pull nomic-embed-text qwen3:0.6b` in builder stage (if Docker embed path)

Env block (copy to `.env` + `.env.example`):

```
# — embeddings (approved) —
MASTYF_AI_EMBEDDING_MODEL=nomic-embed-text
MASTYF_AI_EMBEDDING_THRESHOLD=0.94
# — fast gate (local) —
MASTYF_AI_DISTILLED_MODEL=qwen3:0.6b
MASTYF_AI_SEMANTIC_SYNC_REQUEST_TIMEOUT_MS=800
MASTYF_AI_SEMANTIC_DISTILLED=true
# — heavy analysis (cloud primary, local fallback) —
MASTYF_AI_SAST_MODEL=qwen3-coder:480b-cloud
MASTYF_AI_VULN_ANALYSIS_MODEL=qwen3-coder:480b-cloud
MASTYF_AI_LLM_FALLBACK_MODEL=qwen3:8b
# — cloud burst (async/disclosure only) —
# set ANTHROPIC_API_KEY + OPENAI_API_KEY + ollama signin for 480b-cloud
MASTYF_AI_SEMANTIC_LLM_MAX_PER_MIN=60
MASTYF_AI_SEMANTIC_LLM_MAX_USD_PER_MIN=0.12
MASTYF_AI_LLM_CACHE=true
MASTYF_AI_LLM_CACHE_TTL_SEC=86400
```

10. Verification & Acceptance Criteria

Check	Command	Pass
Sync p95 hot path	`pnpm bench:semantic -- --filter="privilege_escalation"`	`p95 < 120ms` (was 600-2500ms) with distilled+vector on
ADB with semantic on	`MASTYF_AI_LLM_ENABLED=true pnpm test:bench -- AgentDefense` or `node ./adversarial-harness/run.mjs --bench ADB`	`AD 60.2% -> >78%`, `privilege 0%->>70%`, FP `0%`
Async LLM call reduction	`grep embedding_hit_total` in metrics after 1k audits	`-70%` LLM calls after warmup
Vuln disclosure	`vuln run` on canary MCP server	citation ratio `>=50%` (`vuln-analyst.ts:226`), `validate 2-of-3` pass, zip has `final.md` not template
Queue drain concurrency	`tests/ai/semantic-burst-rate-limit.test.ts` + enqueue 50, `drainQueue < 15s`	4x speedup
Cost envelope	`METER semantic-llm-rate-limit`	async `$0.12/min` not exceeded, circuit `5/60s` trips as designed
Fallback correctness	`OLLAMA_BASE_URL=http://127.0.0.1:9999 pnpm test:unit`	heuristic fallback green, no unhandled throw

11. Timeline, Team, Budget, Risk

Timeline (very small team, AI-heavy):

- Phase A 1-2d — immediate wins + 480b-cloud wire (high ROI, zero weight)
- Phase B 2-3d — vector cache `nomic-embed-text`
- Phase C 1-2w — distilled `0.6b` fast gate (core detection fix)
- Phase D parallel — 480b-cloud + cloud burst + Threat Lab batching

Team: Founder (full-time, claims integrity owner) — not substitutable. Part-time security researcher 4-8h/wk *now* (reviews labels, threat model) — highest leverage, equity/hourly. Academic PI after E1/E2 artefacts. AI/ML engineer post-funding. Budget `€200-500/mo` API burst (`haiku $0.25` primary, `480b-cloud $0.15/M`) + `€0` local (`qwen3:0.6b` 0.4GB, `nomic-embed-text` 274MB), free-tier CI, `€8-20k` one-off pentest deferred.

Risk & mitigation:

- 0.6b underperforms on jailbreak framing -> mitigate: category hint + uncertain promote to 8B (3-5% traffic); fallback env `MASTYF_AI_DISTILLED_MODEL=qwen3:8b`.
- `ollama signin` 480b auth expires -> fallback chain `480b-cloud -> qwen3:8b -> haiku` already in `AgenticModelProvider`.
- Embedding drift (0.94 threshold) -> tune via `bench:semantic` ROC on 615 labels; soft hit `0.88-0.94` still distills.

Do NOT do yet (audit §XVI): dashboard/cloud feature expansion, multi-tenant hardening, VS Code, MTX, federated/MPC, more auto-generated corpus — they expand attack surface before evidence exists.

---

12. Appendix — File Inventory

`src/ai` 66 files, `src/vuln-discovery` 27, `security-swarm` 13 agents, `packages/core/src/ai` 6, `src/proxy` 38, `src/policy` 45, `src/agentic/model-provider.ts`, `src/audit/*`, `src/auth/*`, `src/config/llm-config.ts`, `src/tenant/tenant-semantic-config.ts`, `src/services/tenant-budget.ts`, `src/services/cost-auditor.ts`, `src/utils/semantic-timeout.ts`, `src/utils/semantic-layer.ts`. Tests: `tests/ai/*` 20+ (circuit, rate-limit USD, llm-cache LRU, threat-lab, red-team-personas, transport parity).

End of Blueprint — Ready for Phase A execution.

Generated 28 Aug 2026 from read-only deep dive of 278 files. Dashboard `http://localhost:4000` (`--no-apply-ide` to avoid 256-file `launchctl` overflow). Next: `ollama pull nomic-embed-text qwen3:0.6b && ollama signin && pnpm build` then Phase A diffs.